

ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Ingeniería Computacional

Laboratorio de Fundamentos de Electricidad y Sistemas Digitales



Práctica #6: Control por Lógica Combinatoria y Arduino

Implementación y contraste de actuadores con compuertas lógicas y sistemas embebidos

Matrícula	Nombre Completo
202501540	SÁNCHEZ SAAVEDRA FREDDY STEVEN

Guayaquil, Ecuador
9 de agosto de 2026

1. Objetivos

- Implementar el control de actuadores y cargas DC a través de compuertas lógicas como una etapa de control; a través de la implementación de sensores y circuito armado.
- Analizar la importancia de una etapa de acondicionamiento para los IC de compuertas lógicas, para un correcto funcionamiento de los integrados; a través de la revisión de los datasheets que otorgan los fabricantes.
- Comparar la funcionalidad de control implementado con compuertas lógicas versus un control a través de un sistema embebido como Arduino.

2. Materiales

- Fuente de voltaje DC y 1 multímetro.
- U1: LM358, Integrados: 74LS04, 74LS08, 74HC32.
- Diodos: Leds (verde, amarillo, rojo), 1N4007, Zener 1N4733A.
- Transistores: MOSFET IRF640, 2N2222.
- Potenciómetros de 1k o 10k, Resistencias: 220, 330, 560, 1k y 10k Ω .
- 1 motor DC, Arduino Uno o Arduino Nano.

3. Esquemas de Circuitos (Simulaciones e Implementación Física)

Procedimiento 1: Acondicionamiento de Señales

Diseño del bloque comparador con amplificadores operacionales LM358, ajustando los potenciómetros RV1 al 70 % y RV3 al 30 % para controlar la activación de las salidas.

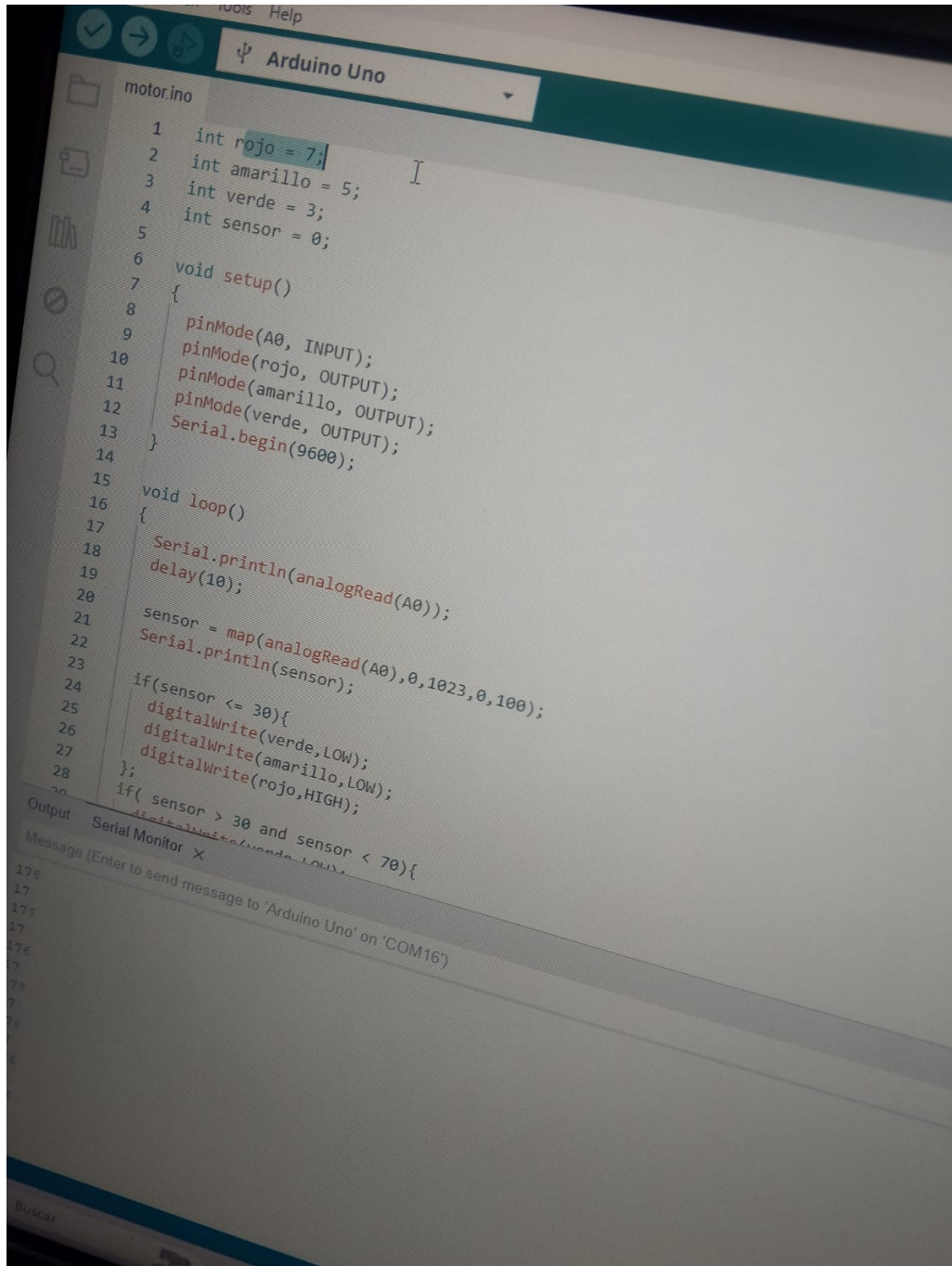


Figura 1: Simulación del bloque comparador y acondicionamiento en Proteus.

Procedimiento 2 y 3: Lógica Combinatoria e Integración con Arduino

Implementación física del circuito de control conectado al Arduino Uno para la lectura analógica y accionamiento lógico de actuadores.

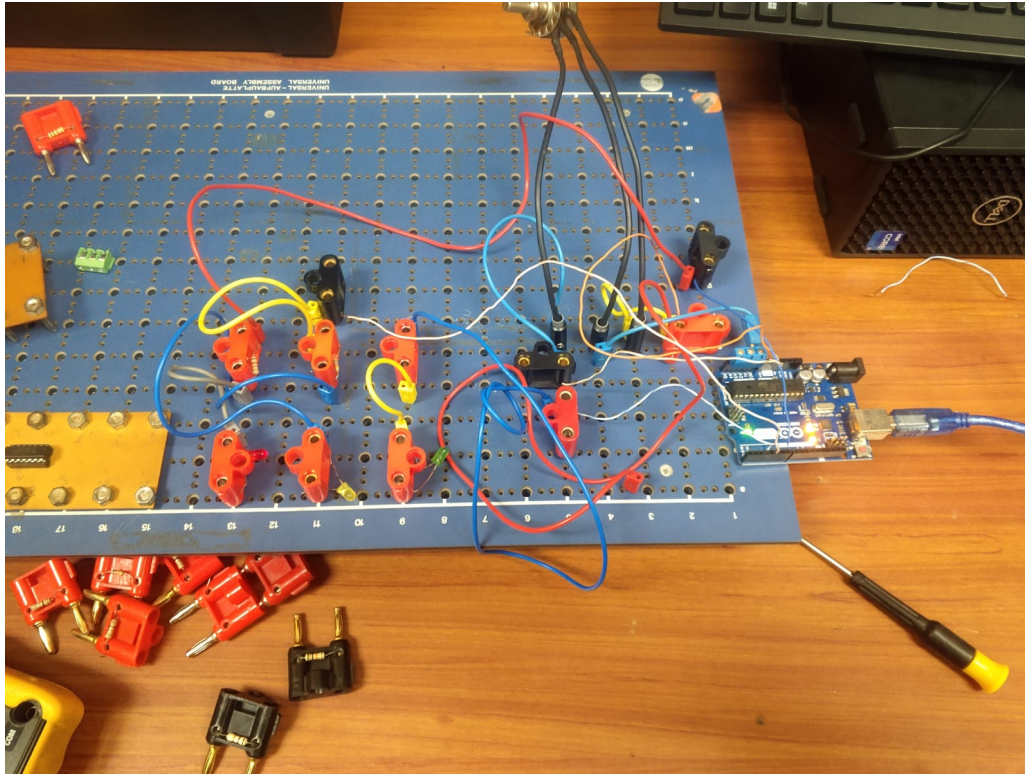


Figura 2: Montaje físico del circuito combinatorio y conexión con la placa Arduino Uno.

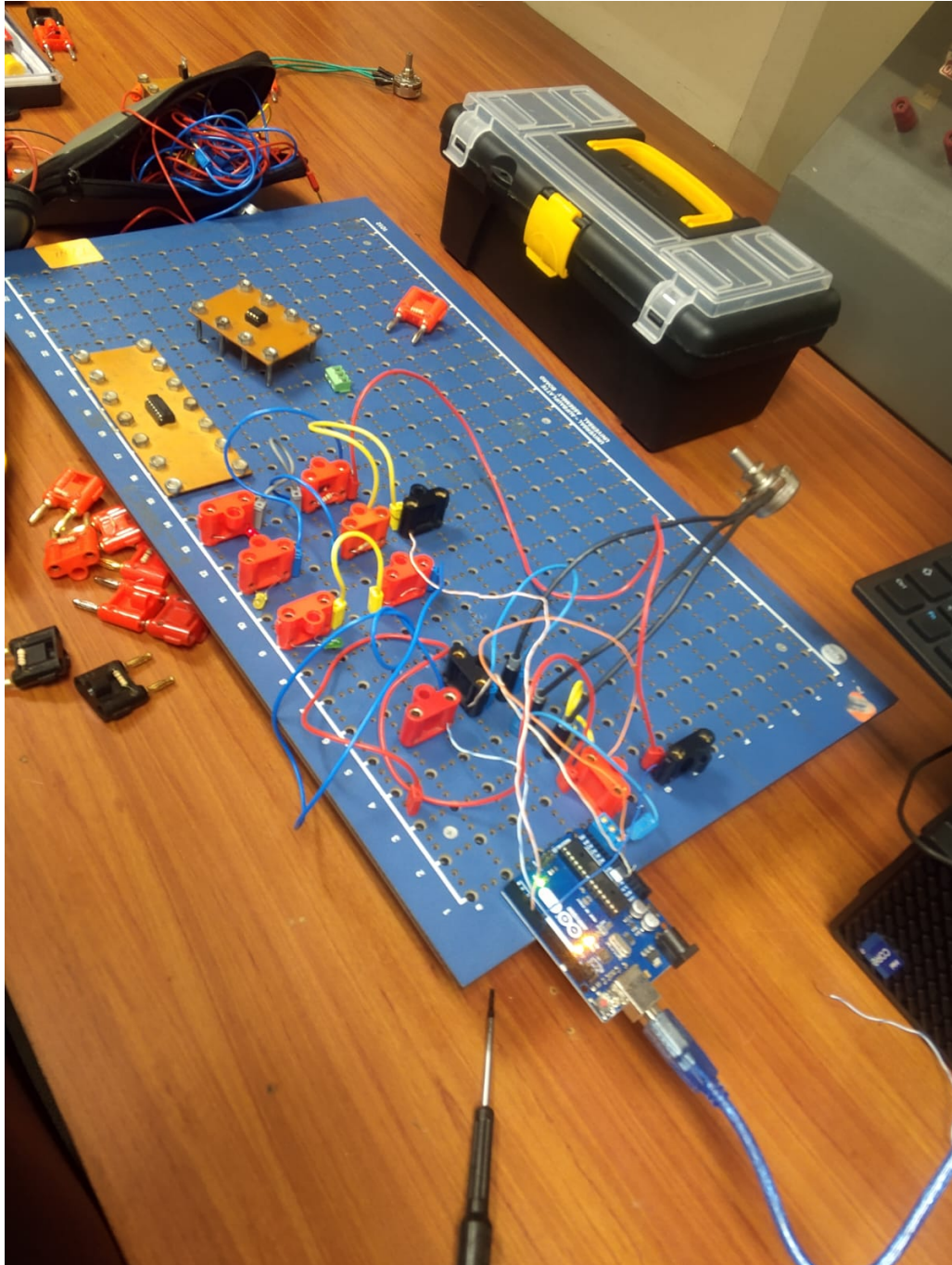


Figura 3: Perspectiva ampliada del laboratorio, evidenciando el cableado del sensor y la etapa de control.

4. Tabla de Resultados

Procedimiento 1

1. ¿Cuál es la función del diodo ZENER 1N4733A?

El diodo Zener 1N4733A (que tiene un voltaje de ruptura de 5.1V) se encarga de regular y fijar el voltaje a un máximo de 5.1V. Su función es proteger las entradas lógicas siguientes, asegurando que las señales acondicionadas no sobrepasen el voltaje de operación seguro de las compuertas (típicamente 5V) y garantizando un nivel lógico "HIGH".^{estable}.

2. ¿Cuáles son los valores de voltaje que se encuentran en RV1 y RV3 para que se activen las salidas cuando varía la señal Vacond?

Al estar configurados como divisores de tensión con una fuente de 12V, teóricamente: RV1 (al 70 %) establece un voltaje de referencia de 8.4V, y RV3 (al 30 %) establece un voltaje de referencia de 3.6V. Las salidas se activarán al cruzar estos umbrales generados por la señal central.

Procedimiento 2

Ecuación deducida del circuito de la Figura 3: $F = A' + (B \cdot C)$

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Cuadro 1: Tabla de verdad obtenida del análisis lógico.

- 1. SOP (Suma de Productos): $F = A'B'C' + A'B'C + A'BC' + A'BC + ABC$
- 2. POS (Producto de Sumas): $F = (A' + B + C) \cdot (A' + B + C') \cdot (A' + B' + C)$
- 3. Mapa de Karnaugh: Simplificado resulta en la función original $F = A' + BC$

Procedimiento 3

Añadir el código modificado para obtener la lectura de voltaje y control de LEDs.

```
// Declaracion de pines y variables
int rojo = 7;
int amarillo = 5;
int verde = 3;
int sensor = 0;
int valorLectura = 0;

void setup() {
  pinMode(A0, INPUT);
  pinMode(rojo , OUTPUT);
  pinMode(amarillo , OUTPUT);
  pinMode(verde , OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // Optimizacion: Leemos el sensor analogico una unica vez por ciclo
  valorLectura = analogRead(A0);
  Serial.println(valorLectura);
  delay(10);

  // Mapeamos el valor leido (0-1023) a un rango de 0-100
  sensor = map(valorLectura , 0, 1023, 0, 100);
  Serial.println(sensor);

  // Logica de control para activacion de salidas
  if(sensor <= 30){
    digitalWrite(verde , LOW);
    digitalWrite(amarillo , LOW);
    digitalWrite(rojo , HIGH);
  }
  else if(sensor > 30 && sensor < 70){
    digitalWrite(verde , LOW);
    digitalWrite(amarillo , HIGH);
    digitalWrite(rojo , LOW);
  }
  else if(sensor >= 70){
    digitalWrite(verde , HIGH);
    digitalWrite(amarillo , LOW);
    digitalWrite(rojo , LOW);
  }
}
```

5. Preguntas

1. Explique la necesidad de acondicionar las señales en voltajes entre 0V a 5V, para las compuertas lógicas.

Las familias lógicas TTL y CMOS (como los integrados 74LS y 74HC utilizados) están diseñadas para operar estrictamente dentro de márgenes definidos (típicamente $V_{cc} = 5V$). Acondicionar la señal es obligatorio por dos motivos: si el voltaje supera los 5V, se pueden perforar o quemar físicamente los transistores internos del IC; y si el voltaje es inferior a los umbrales de detección, el IC no reconocerá correctamente la señal como un "1" lógico (HIGH).

2. Realice un contraste en implementar un controlador con compuertas lógicas versus la misma aplicación en Arduino.

- **Compuertas lógicas (Hardware):** Su respuesta es casi instantánea (retardos de nanosegundos), ideal para operaciones críticas en tiempo. Sin embargo, su escalabilidad es nula; si se necesita cambiar la lógica de control, es obligatorio rediseñar y recablear todo el circuito físico, lo que aumenta costos espaciales y económicos.
- **Arduino (Software embebido):** Ofrece una tremenda flexibilidad. Cambiar la lógica o añadir funcionalidades adicionales (como temporizadores o lecturas analógicas) solo requiere modificar líneas de código, ahorrando gran cantidad de espacio físico (menos integrados en placa). Como desventaja, añade una ligera latencia debido al ciclo de reloj y procesamiento del microcontrolador.

3. Investigar para la sintaxis de las siguientes funciones en Arduino, y explique su funcionalidad.

- **`analogRead(pin)`:** Lee el voltaje aplicado a un pin analógico y lo convierte en un número entero mediante el ADC de 10 bits del Arduino. Retorna un valor entre 0 y 1023 (donde 0 es 0V y 1023 es 5V).
- **`digitalWrite(pin, value)`:** Escribe un valor lógico HIGH (5V) o LOW (0V) en un pin digital que previamente haya sido configurado como salida (OUTPUT).
- **`Serial.begin(speed)`:** Inicializa la comunicación serial del Arduino y establece la tasa de transmisión de datos en bits por segundo (baudios), típicamente 9600. Es esencial para enviar datos al Monitor Serial.
- **`Serial.plot()` / `Serial.println()`:** Envía datos al puerto serial. `println` los imprime como texto con un salto de línea. El "Serial Plotter" (IDE de Arduino) es una herramienta que toma estos números impresos y los grafica en tiempo real en un eje cartesiano, excelente para visualizar variaciones de sensores.